

Mayhem clone

Prosjekt laget i python ved hjelp av pygame

1. Introduksjon

Denne oppgaven tar for seg design, implementasjon og debugging av prosjektet Mayhem clone for faget INF-1400 hos UIT. Vi vil se nærmere på hvordan arkitekturen ser ut for det ferdige prosjektet, hvordan klasse implementasjonen og ansvarsfordeling er, og til slutt hvordan problemer vi støttet på under utviklingen.

2. Arkitektur og filstruktur

2.1 Filer

Prosjektet består av følgende filer:

- Config.py - Denne inneholder konfigurasjonskonstanter
- Game.py - Hovedspillklasse, denne eier game loop, spritegrupper og kollisjonshåndtering
- Spaceship.py – Denne lager Spaceship sprite og håndterer input, fysikk, skyting og krasj/respawn
- Bullet.py – Denne lager bullet sprite, bevegelse og levetid for bullet
- Obstacles.py – Denne lager obstacle og landingpad sprite

2.2 Klasse hierarki

Alle synlige objekter arver fra `pygame.sprite.Sprite`

Klasse	Arver fra	Rolle
Spaceship	Pygame.sprite.Sprite	Spillerkontrollert romskip
Bullet	Pygame.sprite.Sprite	Prosjektil fra romskip
Obstacle	Pygame.sprite.Sprite	Statisk obstacle
Landingpad	Pygame.sprite.Sprite	Landingsplattform for refuel

2.3 Komposisjon

For å gjøre prosjektet lett og oversiktlig har hoved Game kalssen eierskap over alle sprite gruppene og de to spaceship objektene

```
pygame.init()
self.screen = pygame.display.set_mode((config.SCREEN_WIDTH, config.SCREEN_HEIGHT))
pygame.display.set_caption(config.TITLE)
self.clock = pygame.time.Clock()
self.font = pygame.font.SysFont("monospace", 16)
self.running = True

# Sprite groups
self.all_sprites = pygame.sprite.Group()
self.ships = pygame.sprite.Group()
self.bullets = pygame.sprite.Group()
self.obstacles = pygame.sprite.Group()
self.landing_pads = pygame.sprite.Group()

self._setup_world()
self.hud = HUD(self.font)
```

3. Klasse implementasjon

3.1 Fysikk og bevegelse

Romskipene bruker vektorbasert fysikk som er implementert med `pygame.math.Vector2`. (ref: <https://www.pygame.org/docs/ref/math.html>) Dette gjør at vi kan lette implementere tyngdekraft som en multiplikator som skalerer hastigheten hver frame. Dette gjør at vi gradvis bremser vertikal fart og simulerer luftmotstand i stenden for en ren gravitasjon. Thrust legger da til akselerasjon i retningen som romskipet peker:

```
if thrusting:
    rad = math.radians(self.angle)
    thrust_vec = pygame.math.Vector2(math.sin(rad), -math.cos(rad))
    self.vel += thrust_vec * config.THRUST_POWER
    self.fuel -= config.FUEL_BURN_RATE
```

Posisjon oppdateres da med Euler-integrasjon der konstanten `self.pos` er lagret som en vektor. Dette gjort at vi kan plusse på fartsvektoren, `pos += vel`, per frame.

3.2 Rotasjon

Romskipet tegens som en trekant på en SRCALPHA-overflate. Hvert fram roteres `base_image` med `pygame.transform.rotate(-angle)` for korrekt orientering. Rotasjonsvinkelen oppdatere med en konstant fra config filen kal `ROTATION_SPEED`.

3.3 Skyting og kulehåndtering

`Spaceship._try_fire()` avfyrrer med to begrensinger. Den kan kun ha 5 kuler per spiller samtidig, og en cooldown på 300 ms mellom skudd. Kula spawner slik:

```
direction = pygame.math.Vector2(math.sin(rad), -math.cos(rad))
spawn = self.pos + direction * 35
bullet = Bullet(spawn, direction, self.player_id)
bullet_group.add(bullet)
all_sprites.add(bullet)
```

Da vi ganger med 35 for å kunne spawn 35 pixler fra sentrum av spillern. Den får da hastighet i romskipets retning som vi da ganger med BULLET_SPEED fra config filen. Kuler fjernes etter BULLET_LIFETIME er telt ned, eller hvis kula kolliderer med et objekt.

3.4 Kollisjonshåndtering

All kollisjonshåndtering skjer i `Game._check_collisions()` som kalles hver frame.

- Kuler - hindringer: `groupcollide()` fjerner kula, hindringen beholdes
- Kuler - romskip : `spritecollide()` sjekker at kun andres kuler gjør skade, scorer +3 til skytter, krasj for offer
- Romskip – hindringer krasj hvis fart > 0.1, ellers dempning
- Romskip – landingsplattform: `refuel()` kalles
- Romskip – romskip: begge krasjer hvis relativ fart > 1.0

Krasj trekker 1 poeng og kaller `_respawn()` som tilbakestiller posisjon, fart og drivstoff

4. Designmønstre

4.1 Composite / Group mønster

`Pygame.sprite.Group` fungerer som et Composite mønster. Game klasse behandler alle sprites uniformt via `group.update()` og `group.draw()` uten å trenge å kjenne til de individuelle typene. Dette er et eksempel på polymorfisme, siden `update` metoden kalles på alle sprites, men hver klasse implementerer den ulikt.

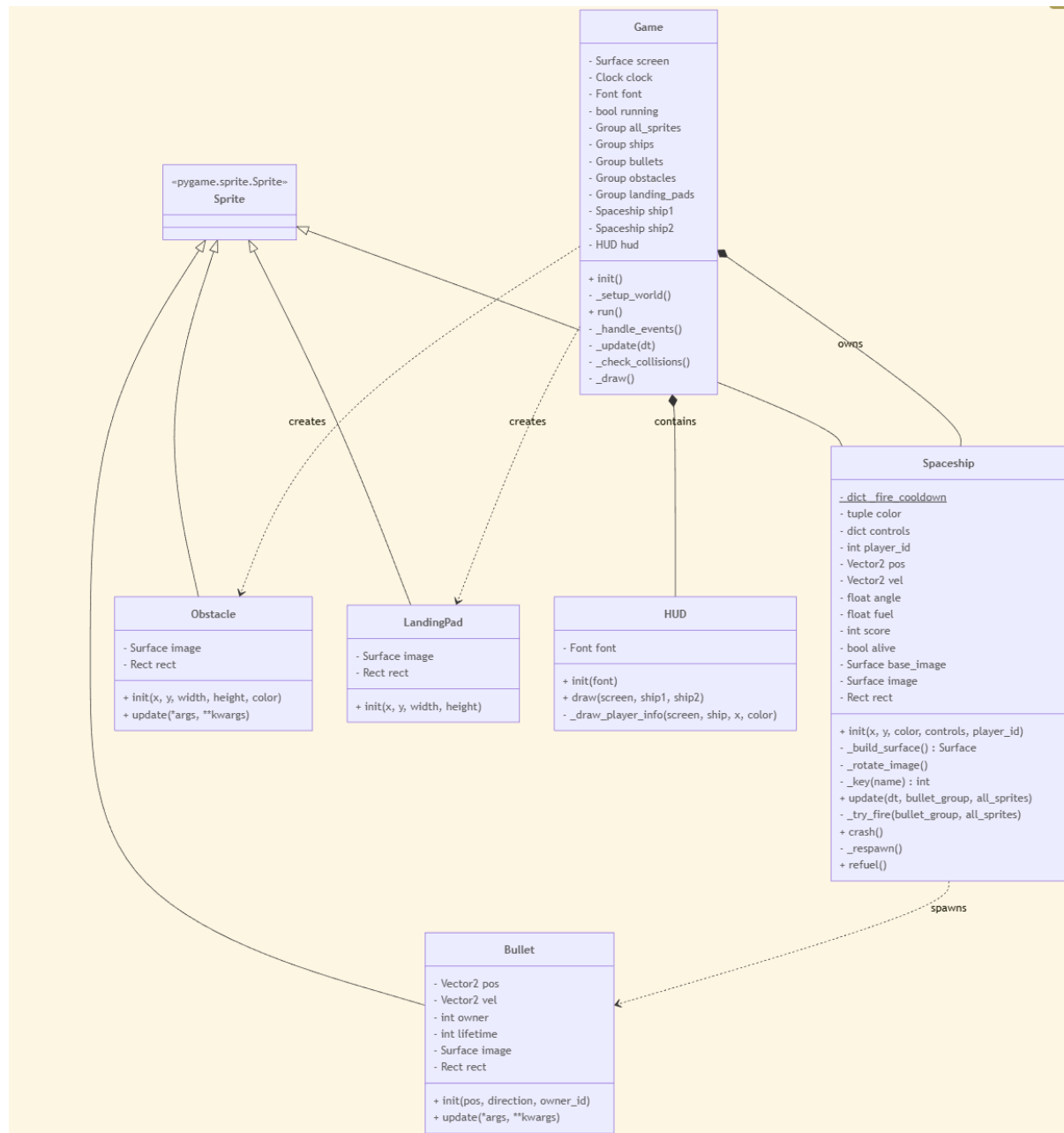
4.2 Konfigurasjon

`Config.py` fungerer som en global konfigurasjon, og dette er da et eksempel på singletons (ref: <https://www.digitalocean.com/community/tutorials/gangs-of-four-gof-design-patterns>) Dette sentraliserer spillparametere og gjør tuning enkelt uten å endre spillogikk.

4.3 Template metode

`Sprite` arv fram `pygame.sprite.Sprite` definerer en kontrakt, ved at alle subklasse mp ha `self.image` og `self.rect`. Game klassen utnytter denne kontrakten gjennom `Group.update()` og `Group.draw()`

5. Klassediagram

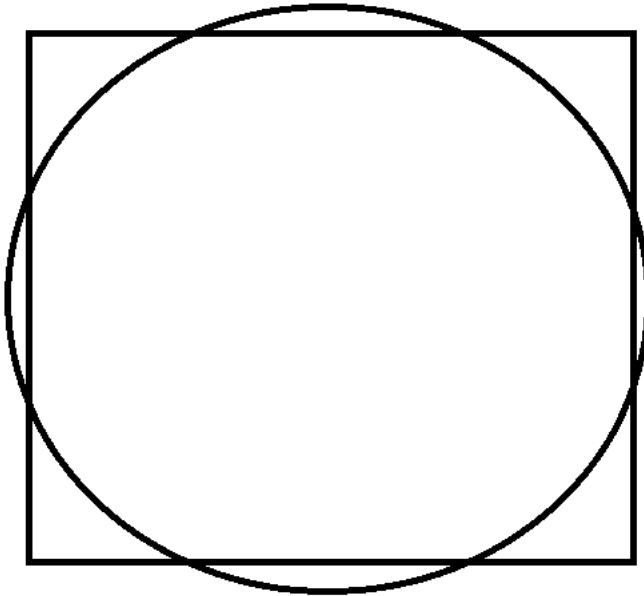


6. Problemer og eksperiment

Et problem vi møtte på under utviklingen av spillet var at bullet ikke kunne skytes fra begge spillerne i visse vinkler. Det vi la merke til var at dette var på et inkrement på 20-40 grader plus 90*n vinkler. En hypotese vi hadde var at game loopen bare hadde mye støy, men etter testing stemmet ikke dette siden kollisjon fungerte og andre mekanikker som thrust og refuel også fungerte.

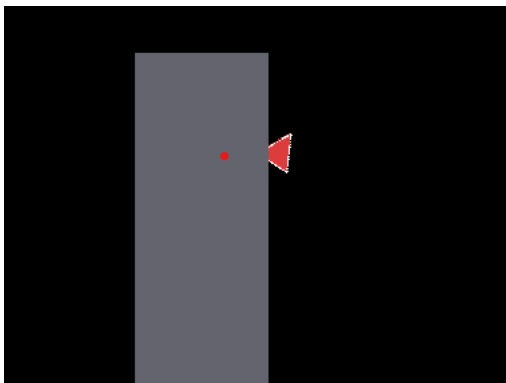
Vi lagde da en illustrasjon over hvordan hitboxen så ut, og hvordan bullet ble generert.

Firkanten er hitbox, sirkel er bullet spawn radius.



Dette ga oss en klar forståelse på at problemet var kun at bullet ble generert inni vår egen hitbox. Når vi da sjekker kollisjon hver tick så viste det seg bare at bullet traff et objekt og mister da sin livsyklus før den rekker å faktisk skyte. Dette løste vi enkelt ved å gjøre sirkelen større slik at sirkelen treffer kantene på firkanten.

Et annet problem vi støttet på var når vi tweaket player movement slik at vi ikke hadde gravitasjon, men heller en slags flytende bevegelse. Dette funket mye bedre for gameplay og var så enkelt som å gange fartsvektoren med en Gravity konstant hver gameplay tick. Buggen vi fant da var at spillerene kunne presse seg gjennom objektene slik at de kunne sitte inni veggen og skyte utover.



Løsningen på buggen fant vi fort ut var her:

```
# Ships hit obstacles
for ship in self.ships:
    if pygame.sprite.spritecollideany(ship, self.obstacles):
        if ship.vel.length() > 1.5:
            ship.crash()
        else:
            ship.vel.y = 0
            ship.vel.x *= 0.8
```

Ved å gi kollisjonen for mye akseptabel verdi kunne man skli inn i veggen ved lav hastighet og deretter presse seg videre ved hjelp av else blokken. Vi endret da kun `ship.vel.length() > 1.5` -> `ship.vel.length() > 0.1` og problemet var da løst.

7. Kjente limitasjoner og ytelse

Spillet har fremmedes mange bugs som og gameplay design som ikke er optimalt. Programmet sin ytelse er også noe å se på ved at vi ikke har mange antall sprites. Hvis vi skulle skalere og håndtere for eksempel flere spillere, bevegelige objekter, eller power-ups måtte vi se nærmere på `pygame.transform.rotate()`. Denne blir kalt hver frame for begge romskipene og krever da at vi må oprette en ny overflate. En løsning kunne da ha vært å bufre roterte bilder.

8. Bruk av ai

Ai har blitt brukt til visse debugging problemer under utviklingen av dette programmet. Alle chat logger blir lagt ved i et eget .txt dokument under doc/ mappen.

Kode er ikke kopiert ifra AI modellen og heller ikke brukt under skriving av rapporten.

9. Konklusjon

Dette prosjektet ble implementert med en tydelig ansvarsfordeling for klassene. Vi ser at tidligere obliger hjalp til her der sprites ble en stor del av designet vårt. Under utviklingen møttet vi også på stadige bugger. Alle ble desverre ikke tatt med i rapporten, utenom de vi var mest fornøyde med. Spillet er da spillbart og demonstrerer rett bruk av arv, polymorfisme og innkapsling i python.